



LUMI AI Factory Service Center
Empowering Europe's AI Ecosystem

Quantum x Finance Hackathon Quickstart

1. Running Quantum Jobs on Q50 via LUMI

Event participant guide — accessing the Q50 quantum computer through the LUMI supercomputer

This guide walks you through running quantum programs on **VTT Q50**, a superconducting quantum computer, from the **LUMI** supercomputer. For this event we have prepared a dedicated software module and a LUMI reservation so you can get started quickly — no prior HPC or quantum experience is required. VTT has also provided an exclusive booking for the hackathon for VTT Q50 access through LUMI.

NOTE

Two things have been set up for you:

1. A custom module, **fiqci-vtt-qiskit-QxF**, with Qiskit and the Q50 tooling pre-installed.
2. A reservation, **quantumfinance**, that gives you priority access to CPU compute during the event.
3. A reservation, **quantumfinance-g**, that gives you priority access to GPU compute during the event.

1.1 Before you begin

- A **MyCSC / Haka** account with access to the event project.
- A web browser. Everything runs in the browser — nothing to install on your own machine.
- These are the values used throughout this guide:

Setting	Value
CPU reservation	quantumfinance
GPU reservation	quantumfinance-g
Project	project_462001763
Partition	small (max walltime 3 days, quantum jobs limited to 2 hours)
Modules to load/use	/appl/local/quantum/modulefiles/hackathon, fiqci-vtt-qiskit-QxF

1.2 Step 1 · Log in and open Jupyter

1. Go to the [LUMI web interface](#) and sign in with your **MyCSC / Haka** account.
2. On the dashboard, find the **Pinned Apps** panel and click **Jupyter**.



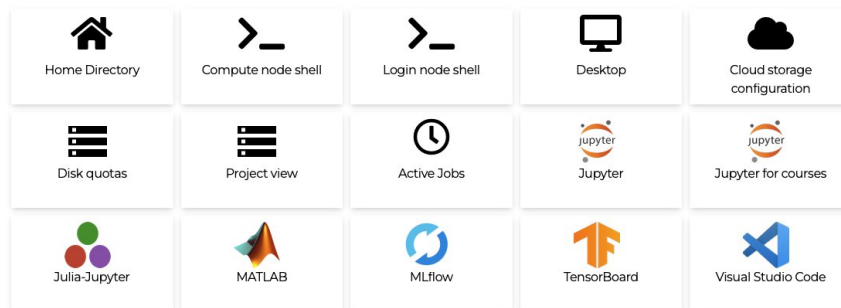
EuroHPC
Joint Undertaking



The LUMI AI Factory Service Center is funded jointly by the EuroHPC Joint Undertaking under Grant Agreement 101234208 and the Participating States FI, CZ, DK, EE, NO, PL.

Funded by the European Union. Views and opinions expressed are however those of the author(s) only and do not necessarily reflect those of the European Union or EuroHPC Joint Undertaking. Neither the European Union nor the granting authority can be held responsible for them.

Pinned Apps



The LUMI dashboard — select the “Jupyter” app.

1.3 Step 2 • Configure your session

On the Jupyter launch form, set the following fields:

- **Reservation** → choose **quantumfinance**.
- **Project** → select the event project (**project_462001763**).
- **Time** → up to **72:00:00**. The reservation uses the **small** partition, which has a limit of 3 days of walltime per session. Quantum jobs have a separate limit of **2:00:00**
- **Working directory** → set this to your scratch directory and create a subfolder for your work (for example **/scratch/project_462001763/yourname**). This is where your notebooks will be saved.
- **Script or path to script** → paste the two module-load lines below.

```
module use /appl/local/quantum/modulefiles/hackathon
module --ignore_cache load fiqci-vtt-qiskit-QxF
```

Saved Settings

You have no saved settings.

Apps

Editors

Visual Studio Code

Graphical applications

Desktop

Servers

MLflow

Julia-Jupyter

MATLAB

TensorBoard

Marimo Notebook

Jupyter

Tools

Compute node shell

Course environments

Jupyter for courses

Jupyter

Interactive Jupyter session

[Documentation](#)

Reservation

No reservation

Project

project_462001763 (Quantum x Finance Hackathon 2026)

The selected project will be available as **\$PROJECT** in other fields in this form.

Partition

small

Resources

Number of CPU cores

1

Memory (GiB)

1

Time

4:00:00

d-hh:mm:ss, or hh:mm:ss

Settings

Working directory

/users/nivalajo

Basic

Advanced

Custom Python type

Script

Script or path to script

```
module use /appl/local/quantum/modulefiles/hackathon
module --ignore_cache load fiqci-vtt-qiskit-QxF
```

The provided script will be run before starting Jupyter. If a path to a script is provided, it will be sourced by bash before starting Jupyter. Ensure that the script sets up the environment so that **python** is in **PATH** and Jupyter is installed.

☐ Save settings

Save settings and close

Launch

Reset to default settings

The launch form — set the reservation, project, time, working directory and module lines.

TIP

Want GPU access, use the **small-g** partition and **quantumfinance-g** reservation. For a larger node Set **Reservation** to **No reservation** and pick a different partition. See [LUMI partitions](#) for the options. Note that jobs outside the reservation may queue for longer.

1.4 Step 3 • Launch and connect

1. Click the blue **Launch** button at the bottom of the form.



EuroHPC
Joint Undertaking



The LUMI AI Factory Service Center is funded jointly by the EuroHPC Joint Undertaking under Grant Agreement 101234208 and the Participating States FI, CZ, DK, EE, NO, PL.

Funded by the European Union. Views and opinions expressed are however those of the author(s) only and do not necessarily reflect those of the European Union or EuroHPC Joint Undertaking. Neither the European Union nor the granting authority can be held responsible for them.

Scroll to the bottom of the form and click “Launch”.

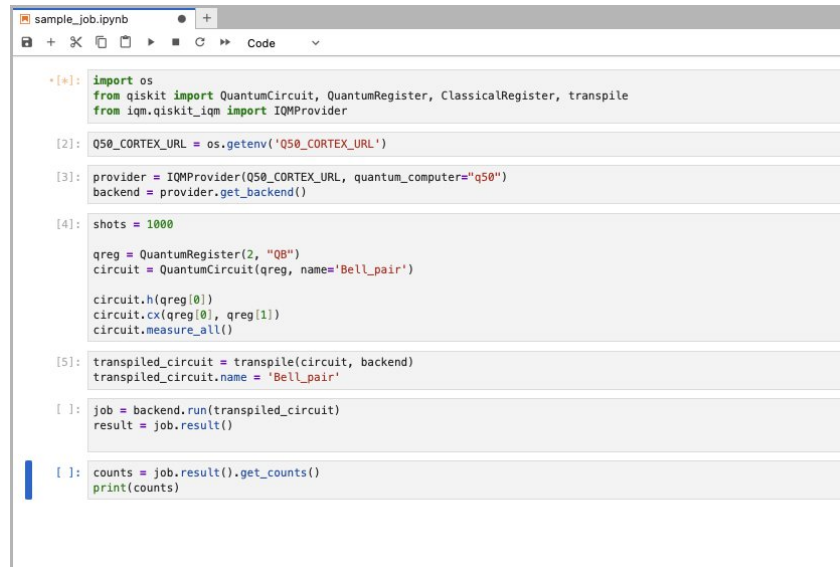
2. Your session is now queued. Depending on demand this takes a few seconds to a few minutes.
3. When the card turns **Running**, click **Connect to Jupyter**. JupyterLab opens in a new browser tab.

The screenshot displays the LUMI AI Factory web interface. On the left, a sidebar menu lists various applications and tools, including Visual Studio Code, Desktop, MLflow, Julia-Jupyter, MATLAB, TensorBoard, Marimo Notebook, Jupyter, and Compute node shell. The main panel shows a Jupyter session (ID: 22549177) in a 'Running' state with 1 node and 2 cores. It provides details such as the host address, creation time, time remaining (57 minutes), session ID, and project information. A 'Connect to Jupyter' button is prominently displayed at the bottom of the session card.

A running session — click “Connect to Jupyter” to open your notebook.

1.5 Step 4 • Run your first quantum program

In JupyterLab, create a new notebook and try the example below. It builds a **Bell pair** — a simple two-qubit entangled state — runs it on Q50, and prints the measurement results.



```

[*]: import os
    from qiskit import QuantumCircuit, QuantumRegister, ClassicalRegister, transpile
    from qiskit_ismq import IQMProvider

[2]: Q50_CORTEX_URL = os.getenv('Q50_CORTEX_URL')

[3]: provider = IQMProvider(Q50_CORTEX_URL, quantum_computer='q50')
    backend = provider.get_backend()

[4]: shots = 1000

    qreg = QuantumRegister(2, 'QB')
    circuit = QuantumCircuit(qreg, name='Bell_pair')

    circuit.h(qreg[0])
    circuit.cx(qreg[0], qreg[1])
    circuit.measure_all()

[5]: transpiled_circuit = transpile(circuit, backend)
    transpiled_circuit.name = 'Bell_pair'

[ ]: job = backend.run(transpiled_circuit)
    result = job.result()

[ ]: counts = job.result().get_counts()
    print(counts)

```

A sample notebook that runs a Bell-pair circuit on Q50.

The same code as text, so you can copy it:

```

import os
from qiskit import QuantumCircuit, QuantumRegister, ClassicalRegister,
transpile
from qiskit_ismq import IQMProvider

# The module sets this environment variable for you
Q50_CORTEX_URL = os.getenv('Q50_CORTEX_URL')
provider = IQMProvider(Q50_CORTEX_URL, quantum_computer='q50')
backend = provider.get_backend()

shots = 1000
qreg = QuantumRegister(2, 'QB')
circuit = QuantumCircuit(qreg, name='Bell_pair')
circuit.h(qreg[0]) # superposition
circuit.cx(qreg[0], qreg[1]) # entangle the two qubits
circuit.measure_all()

transpiled = transpile(circuit, backend)
job = backend.run(transpiled, shots=shots)
job_id = job.job_id() # save the job_id to fetch results later if something
goes wrong
counts = job.result().get_counts()
print(counts)

```

A healthy Bell pair gives mostly 00 and 11 outcomes, with a few 01 / 10 counts caused by hardware noise.

TIP

Save your work from the top-left menu: **File** → **Save Notebook** (one file) or **Save All** (everything). Files are written to the working directory you chose at launch.

IMPORTANT

Save the `job_id` of your quantum job to fetch results later if something goes wrong. To fetch results of a job by `job_id` run `backend.retrieve_job(<job_id>)`.

2. Running jobs from the terminal (sbatch)

Notebooks are great for exploring, but for longer or unattended runs it is often cleaner to submit a batch job. Open a terminal — either **Login node shell** on the dashboard, or **File** → **New** → **Terminal** inside JupyterLab — and write a Slurm script.

2.1 1. Save your program as a .py file

For example, save the example above as `bell.py` in your scratch folder.

2.2 2. Create a job script

Create a file called `run_q50.sh`:

```
#!/bin/bash
#SBATCH --job-name=q50-bell
#SBATCH --account=project_462001763
#SBATCH --reservation=quantumfinance
#SBATCH --partition=small
#SBATCH --time=00:15:00
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --output=q50-%j.out

module use /appl/local/quantum/modulefiles/hackathon
module --ignore_cache load fiqci-vtt-qiskit-QxF

python bell.py
```

2.3 3. Submit and monitor

```
sbatch run_q50.sh      # submit the job
squeue --me           # see your queued / running jobs
cat q50-<jobid>.out    # read the output once it finishes
```

```
scancel <jobid>          # cancel a job if needed
```

NOTE

The `--account`, `--reservation` and `--partition` lines must match the event values for the reservation to apply. Drop the `--reservation` line if you are running outside the event window.

3. Quantum Circuit Simulation on LUMI

3.1 Using CPUs

```
#!/bin/bash
#SBATCH --job-name=aer_cpu_job
#SBATCH --output=aer_cpu_job.o%j
#SBATCH --error=aer_cpu_job.e%j
#SBATCH --account=project_462001763
#SBATCH --time=2:00:00
#SBATCH --partition=small
#SBATCH --reservation=quantumfinance
#SBATCH --nodes=1
#SBATCH --ntasks-per-node=1
#SBATCH --cpus-per-task=16
#SBATCH --mem=28G
#SBATCH --hint=nomultithread

module use /appl/local/quantum/modulefiles/hackathon
module --ignore_cache load fiqci-vtt-qiskit-QxF

export OMP_NUM_THREADS=${SLURM_CPUS_PER_TASK}
export OMP_PLACES=cores
export OMP_PROC_BIND=close

srun --cpu-bind=cores python myprog_cpu.py
```

In your python script use your preferred simulation method e.g. MPS

```
Import os
from qiskit_aer import AerSimulator

n_threads = int(os.environ.get("SLURM_CPUS_PER_TASK", os.cpu_count()))
mem_mb = int(os.environ.get("SLURM_MEM_PER_NODE", 0)) or None
```

```
sim = AerSimulator(
    method="matrix_product_state",
    device="CPU",
    matrix_product_state_max_bond_dimension=1024,
    matrix_product_state_truncation_threshold=1e-12,
    mps_omp_threads=n_threads,
    mps_parallel_threshold=14,
    max_parallel_threads=n_threads,
    max_memory_mb=mem_mb,
)
```

3.2 Using GPUs

For GPU accelerated Qiskit Aer simulations we provide a special container with a GPU ready build. Use the batch script below to use the container.

```
#!/bin/bash
#SBATCH --job-name=aer_job
#SBATCH --output=aer_job.o%j
#SBATCH --error=aer_job.e%j
#SBATCH --account=project_462001763
#SBATCH --time=2:00:00
#SBATCH --partition=small-g
#SBATCH --reservation=quantumfinance-g
#SBATCH --nodes=1
#SBATCH --gpus-per-node=8
#SBATCH --ntasks-per-node=1
#SBATCH --cpus-per-task=56

export LUMI_QISKIT_SINGULARITY_CONTAINER_PATH=
/app1/local/quantum/qiskit/qiskit_2.3.0_csc.sif
export WRAPPER_PATH=/app1/local/quantum/qiskit/run-singularity

echo "NODES                : ${SLURM_NNODES}"
echo "TASKS PER NODE        : ${SLURM_NTASKS_PER_NODE}"
echo "CPUS PER TASK           : ${SLURM_CPUS_PER_TASK}"
echo "GPUS PER NODE           : ${SLURM_GPUS_PER_NODE}"
echo "GPUS PER TASK           : ${SLURM_GPUS_PER_TASK}"

mask=mask_cpu:0xfe000000000000,0xfe000000000000,0xfe0000,0xfe000000,0xfe,
0xfe00,0xfe00000000,0xfe0000000000

export MPICH_GPU_SUPPORT_ENABLED=1
export MPICH_GPU_IPC_CACHE_MAX_SIZE=100
```

```
export MPICH_GPU_IPC_THRESHOLD=524288
export MPICH_OFI_NIC_POLICY=GPU

srun --cpu-bind=$mask $WRAPPER_PATH $LUMI_QISKIT_SINGULARITY_CONTAINER_PATH
python myprog.py
```

For an example python script see https://docs.csc.fi/apps/qiskit/#lumi_1.

4. Installing extra Python packages

The event module already includes Qiskit and the Q50 tools. If you need an additional package, install it into **your own user space** with the `--user` flag so you do not touch the shared system environment.

In a notebook cell:

```
import sys
!{sys.executable} -m pip install --user matplotlib
```

Or from a terminal (after loading the modules):

```
module use /appl/local/quantum/modulefiles/hackathon
module --ignore_cache load fiqci-vtt-qiskit-QxF
pip install --user matplotlib
```

IMPORTANT

Do not use `sudo` and never install system-wide — it will fail and is not allowed on LUMI. Always use `--user`. After installing, restart the notebook kernel (**Kernel → Restart Kernel**) so the new package is picked up.

TIP

Working on a larger project? Consider using [Tykky](#) for building a custom environment or a personal virtual environment in your scratch space (`python -m venv myenv`) to keep dependencies isolated and reproducible.

5. Improving accuracy with FiQCI-EMS

The **FiQCI Error Mitigation Service (EMS)** is a Python library that wraps IQM quantum backends — such as Q50 — and applies **quantum error mitigation** transparently. Real quantum hardware is noisy, so raw results contain errors. EMS lets you run the same circuit with improved accuracy simply by choosing a **mitigation level**, without having to implement the mitigation techniques yourself. See fiqci.fi/fiqci-ems for full details.



EuroHPC
Joint Undertaking



The LUMI AI Factory Service Center is funded jointly by the EuroHPC Joint Undertaking under Grant Agreement 101234208 and the Participating States FI, CZ, DK, EE, NO, PL.

Funded by the European Union. Views and opinions expressed are however those of the author(s) only and do not necessarily reflect those of the European Union or EuroHPC Joint Undertaking. Neither the European Union nor the granting authority can be held responsible for them.

The library is already available in the event module (`fiqci-vtt-qiskit-QxF`). If you are working outside the module, you can install it into your own user space:

```
pip install --user fiqci-ems
```

Using EMS is a small change to the example from Step 4: instead of running on the bare backend, you wrap it with EMS and pass a mitigation level. A higher level applies stronger mitigation (and may take longer):

```
import os
from qiskit import QuantumCircuit, transpile
from iqm.qiskit_iqm import IQMProvider
from fiqci.ems import FiQCIBackend  # wraps an IQM backend

Q50_CORTEX_URL = os.getenv('Q50_CORTEX_URL')
provider = IQMProvider(Q50_CORTEX_URL, quantum_computer='q50')
backend = provider.get_backend()

# Wrap the backend and request a mitigation level
mitigated = FiQCIBackend(backend, mitigation_level=2)

circuit = QuantumCircuit(2)
circuit.h(0)
circuit.cx(0, 1)
circuit.measure_all()

transpiled = transpile(circuit, mitigated)
job = mitigated.run(transpiled, shots=1000)
print(job.result().get_counts())
```

IMPORTANT

The exact class and parameter names (e.g. `FiQCIBackend`, `mitigation_level`) may differ between versions — check the [FiQCI-EMS documentation](#) or run `help(fiqci_ems)` for the current API before relying on this example.

6. FAQ & troubleshooting

How many CPU/GPU/QPU seconds does the project have?

VTT has provided unlimited VTT Q50 QPU access for the duration of the hackathon. From LUMI the initial allocation is 1 million CPU hours and 5000 GPU hours.

My session is stuck in “Queued” / takes a long time to start.

Make sure you selected the `quantumfinance` reservation and the `small` partition for CPU (including QPU jobs) or `quantumfinance-g` reservation and the `small-g` partition — these give you



EuroHPC
Joint Undertaking



The LUMI AI Factory Service Center is funded jointly by the EuroHPC Joint Undertaking under Grant Agreement 101234208 and the Participating States FI, CZ, DK, EE, NO, PL.

Funded by the European Union. Views and opinions expressed are however those of the author(s) only and do not necessarily reflect those of the European Union or EuroHPC Joint Undertaking. Neither the European Union nor the granting authority can be held responsible for them.

priority. Outside the reservation, jobs share LUMI with everyone and may wait longer. Requesting fewer cores and less memory also starts faster.

My session ended and I lost my work.

The small partition allows 3 days per session however quantum jobs have a separate limit of 2 hours. Save often (**File** → **Save All**), and store notebooks in your **scratch** folder rather than the temporary session space. Long-running work is better submitted with **sbatch** (see above).

I get `ModuleNotFoundError` / “No module named qiskit”.

The module lines were probably not loaded. Confirm both `module use /appl/local/quantum/modulefiles/hackathon` and `module --ignore_cache load fiqci-vtt-qiskit-QxF` are in the “Script or path to script” box at launch, then relaunch. In a terminal, run them manually before starting Python.

A package I installed isn’t found in the notebook.

Install with `--user`, then restart the kernel (**Kernel** → **Restart Kernel**). If it still isn’t found, the notebook may be using a different Python — check with `import sys; print(sys.executable)` and reinstall against that interpreter.

My quantum job fails or hangs when I call `backend.run()`.

Q50 may be offline for calibration or maintenance. Check live system status at fiqci.fi/status. Also confirm your circuit was **transpiled** for the backend before running.

Where can I get help during the event?

Ask the CSC/LUMI AI Factory mentors, and include the `output.log` file shown on your session card if you open a support ticket. The links below also have detailed documentation.

7. Useful links

- [CSC — Quantum computing overview](#)
- [CSC — Q50 docs](#)
- [CSC — GPU accelerated Qiskit Aer](#)
- [FiQCI Error Mitigation Service \(EMS\)](#)
- [FiQCI live system status](#)
- [Other FiQCI software tools](#)
- [LUMI Slurm partitions](#)
- [LUMI storage & directories](#)